About this PDF

Built only from: **NOC:Cloud Computing and Distributed Systems (NPTEL course book, week 4)** — https://nptel.ac.in/courses/106104182

Licence: CC BY-SA 4.0. Accessed 2026-09-26.

Covers 8 of 10 syllabus sub-topics of this unit.

Partly covered in the source:

- Suzuki-Kasami’s Broadcast Algorithm
- Deadlock Detection in Distributed Systems: Introduction
- Models of Deadlocks

Not in the source (not covered in this PDF):

- Preliminaries
- Preliminaries
- Chandy-Misra-Haas Algorithm for the AND model and OR Model.

Distributed Mutual exclusion Algorithms: Introduction

Explanation

Central solutions use a master that keeps a queue of waiting requests and a token that allows a process to enter the critical section. The master grants the token and receives it back when the process exits. This approach ensures safety, liveness, and fairness but suffers from a single point of failure and performance bottleneck. Ring based algorithms circulate a single token among processes in a logical ring; a process can enter the critical section only when it holds the token and passes it to its successor after exiting. Lamport's algorithm uses request, reply, and release messages to order critical section entries; an optimization can suppress the release message. In practice, systems such as Google's Chubby use a Paxos-like consensus protocol to handle failures and provide advisory locks.

Example

Google's Chubby system is a distributed lock service that uses a Paxos-like consensus protocol to handle failures and provides advisory locks; clients must explicitly acquire a lock before accessing critical resources. Ring based mutual exclusion is an example where processes form a logical ring and a single token circulates among them; a process can enter the critical section only when it holds the token.

Key points

- Central algorithms use a master that grants a single token to processes, ensuring safety, liveness, and fairness.
- Ring based algorithms circulate a single token among processes in a logical ring, providing safety and liveness but with higher client delay.
- Lamport's algorithm orders critical section entries using request, reply, and release messages, and can be optimized to reduce message count.

- Google's Chubby system uses a Paxos-like consensus protocol to handle failures and offers advisory locks that clients must acquire.
- Failure handling in distributed mutual exclusion can be achieved with Paxos-like schemes, as seen in Chubby and Zookeeper.

Lamport’s algorithm

Definition

Lamport's algorithm is a distributed mutual exclusion algorithm that uses logical clocks to order requests.

Explanation

The algorithm maintains a request queue at each site, ordered by timestamps, and relies on three message types: request, reply, and release. Logical clocks are updated with each message to preserve causality. A site can enter the critical section only when its own request is at the top of its queue and it has received replies from all other sites. When a site exits the critical section it removes its request from the queue and broadcasts a release message, allowing the next request to be processed. The algorithm guarantees fairness: a request with a smaller timestamp will be served before any request with a larger timestamp, ensuring FIFO ordering.

Example

If sites S1 and S2 simultaneously request the critical section, each broadcasts its request. S1 receives S2's request and replies; S2 receives S1's request and replies. After receiving all replies, S1 finds its request (1,1) at the top of its queue and enters the critical section first, followed by S2.

Algorithm / Steps

1. **Requesting the critical section**: broadcast a request message with a timestamp and process id, and enqueue the request locally.
2. **Replying to requests**: upon receiving a request, enqueue it and send a reply back to the requester.
3. **Executing the critical section**: enter only if the request is at the top of the queue and all replies have been received.
4. **Releasing the critical section**: remove the request from the queue and broadcast a release message to all sites.

Key points

- Uses logical clocks to order requests.
- Requires three message types: request, reply, release.
- Guarantees fairness by serving requests in timestamp order.
- Performance: 3(N - 1) messages per critical-section invocation, with a synchronization delay of T.

Ricart

Definition

Ricart Agrawala algorithm is a classical distributed mutual exclusion algorithm that uses Lamport logical clocks, multicast request messages, and reply messages to coordinate access to a critical section.

Explanation

The algorithm works by having each process broadcast a request containing a timestamp. When a process receives a request, it replies immediately if it is not requesting or executing the critical section, or if its own request has a higher timestamp. Otherwise, it defers the reply and records the requester in a deferred array. After a process has received replies from all other processes, it may enter the critical section. Upon exiting, it sends reply messages to any deferred requests and clears the deferred array, allowing waiting processes to proceed.

Example

For instance, consider three sites S1, S2, and S3. S1 and S2 concurrently request the critical section with timestamps. Because S2's timestamp is lower, S2's request is deferred. S1 enters the critical section, and after it exits it sends a reply back to S2, which then enters the critical section.

Algorithm / Steps

1. Requesting the critical section: broadcast a timestamped request to all other sites.
2. Upon receiving a request from Pi, a site Sj replies immediately if Sj is neither requesting nor executing the critical section, or if Sj's own request timestamp is larger than Pi's. Otherwise, Sj defers the reply and records Pi in its deferred request array.
3. After receiving replies from every other site, Pi enters the critical section.
4. When Pi exits the critical section, it sends reply messages to all sites that had deferred requests and clears the corresponding entries in its deferred request array.

Key points

- Uses Lamport logical clocks and a request-deferred array to order requests.
- Replies are deferred when a site's own request has higher priority.
- After all replies are received, a process can enter the critical section.
- Upon exit, deferred replies are sent and the deferred array is cleared, allowing waiting processes to proceed.
- The algorithm requires 2(N-1) messages per critical section invocation and has a synchronization delay of T.

Agrawala’s Algorithm –- Token-Based Algorithms

Definition

Ricart Agrawala algorithm is a request-reply algorithm for distributed mutual exclusion that uses Lamport logical clocks to order requests.

Explanation

When a process wants to enter a critical section it broadcasts a request message containing a timestamp to all other processes. It then waits until it receives a reply from every other process. A process replies immediately if it is neither requesting nor executing the critical section, or if its own request has a higher timestamp than the incoming request. Otherwise it defers the reply and records the request in a deferred array. Once all replies are received, the requesting process enters the critical section. After exiting, it sends replies to any deferred requests.

Example

In an example with three sites, S1 and S2 request the critical section concurrently. S1's request has a lower timestamp, so S2 defers its reply. S1 enters the critical section, exits, and then sends the reply to S2, allowing S2 to enter. The algorithm requires 2(N-1) messages per critical section invocation and has the same synchronization delay as Lamport's algorithm.

Algorithm / Steps

1. **Requesting the critical section** - broadcast a timestamped request to all sites.
2. **Receiving a request** - if not requesting or executing, or if own request has higher timestamp, reply immediately; otherwise defer reply and record the request.
3. **Executing the critical section** - enter after receiving replies from all sites.
4. **Releasing the critical section** - send replies to all deferred requests.

Key points

- Uses Lamport logical clocks for causality.
- Relies on request and reply messages, not a token.
- Requires 2(N-1) messages per critical section invocation.
- Synchronization delay is the same as Lamport's algorithm.

Suzuki-Kasami’s Broadcast Algorithm

Partly covered in the source.

Definition

So, the client send the read request to the master which serves it locally directly. However when a client sends a write request to the master then master will send it to all the servers it will broadcast and wait for the replies. If the majority rep

Explanation

lies this becomes a majority. So, majority is also decided by the size of the quorum. So, if the majority gets the replies or response. So, the master correspondingly response back with the right operations, now you know that if the majority is taken care in the right operation and as far as any two quorums are concerned they are not null as far as

Deadlock Detection in Distributed Systems: Introduction

Partly covered in the source.

Explanation

Deadlock handling in distributed systems must consider failures that earlier algorithms ignored. To address this, Paxos-like consensus protocols are employed, allowing the system to tolerate failures while still solving distributed mutual exclusion. Google's Chubby lock service exemplifies this approach: it uses a Paxos-based scheme to manage failures and provide mutual exclusion through advisory locks, which require clients to explicitly acquire a lock before accessing critical resources. If a client does not obtain the lock, mutual exclusion is not guaranteed, thereby preventing deadlock scenarios that could arise from unsynchronized access. The Chubby architecture consists of a group of servers, one elected as master and the rest as replicas; the master handles read requests locally, while replicas maintain consistency through the consensus protocol.

Example

In the Chubby system, a client wishing to modify a shared configuration file must first request an advisory lock from the master. Only after acquiring this lock does the client perform the write operation, ensuring that no other client can concurrently modify the file. If the master fails, the Paxos-like protocol elects a new master among the replicas, allowing the system to continue functioning without deadlock.

Key points

- Deadlock handling must account for failures that earlier algorithms ignored.
- Paxos-like consensus protocols enable failure-tolerant mutual exclusion.
- Google's Chubby uses advisory locks, requiring explicit lock acquisition for mutual exclusion.
- The Chubby architecture includes a master and replica servers, with the master serving read requests locally.

System Model

Definition

The system model for distributed computing consists of multiple processes that communicate only through message passing, with no shared memory available for direct variable access.

Explanation

Because shared variables cannot be used, mutual exclusion must be achieved by exchanging messages between processes. This model requires that all coordination and synchronization be performed via communication protocols rather than shared state.

Example

In a cloud-based bank server, two ATMs (processes) simultaneously read the same account balance, modify it locally, and write it back to the server. Without proper coordination, this race condition can lead to incorrect final balances. This scenario illustrates the need for mutual exclusion in a distributed system.

Key points

- Processes communicate only by sending and receiving messages.
- No shared memory exists; all coordination must be message-based.
- Mutual exclusion in this model relies on protocols that enforce exclusive access through communication.
- The bank server example demonstrates how concurrent access without coordination can cause data inconsistency.

Models of Deadlocks

Partly covered in the source.

Definition

How to handle the deadlock Maekawa algorithm has also given the provision to handle the deadlock; so, Maekawa hand I will go algorithm handles deadlock by requiring site to yield the lock. If the timestamp of its request is larger than the timestamp

Explanation

of other requests waiting for the same lock [FL]. Handling deadlocks Maekawa's algorithm handles deadlocks by requiring a site to yield the lock if the timestamp of its request is larger than the timestamp of some other request waiting for some lock. That means, it has given a provision of yielding a lock if higher priority process comes and founds

Source and licence

This PDF is adapted from **NOC:Cloud Computing and Distributed Systems (NPTEL course book, week 4)** (https://nptel.ac.in/courses/106104182), accessed 2026-09-26.

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).